

Παράλληλος και Κατανεμημένος Υπολογισμός I

Κατανεμημένη Μνήμη Μεταβίβαση Μηνυμάτων (συν. 1)

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2024



Πελάτης και διακομιστής

Πελάτης και διακομιστής 1

- Διακομιστής

μια διεργασία που (συνήθως) εκτελείται συνεχώς και αναμένει αιτήματα, τα οποία επεξεργάζεται και (συνήθως) τα απαντά. Πάντως τροποποιεί την κατάσταση της (τις μεταβλητές της) με βάση τα αιτήματα που δέχεται.

- Πελάτης

μια διεργασία που αποστέλλει αιτήματα σε ένα διακομιστή, και (συνήθως) αναμένει την απάντηση του διακομιστή για να τροποποιήσει την κατάσταση της.

Πελάτης και διακομιστής 2

- Το βασικότερο μοντέλο επικοινωνίας και αλληλεπίδρασης διεργασιών στο κατανεμημένο υπολογισμό.
- Σχεδόν το σύνολο των κατανεμημένων υπολογισμών εφαρμόζει ένα είδος Πελάτη Διακομιστή.
- Το μοντέλο έχει ευρύτατη εφαρμογή και στο προγραμματισμό κατανεμημένης μνήμης σε ένα υπολογιστικό σύστημα.
- Η έννοια της υπηρεσίας (service) είναι στην ουσία συνώνυμη του διακομιστή (server).

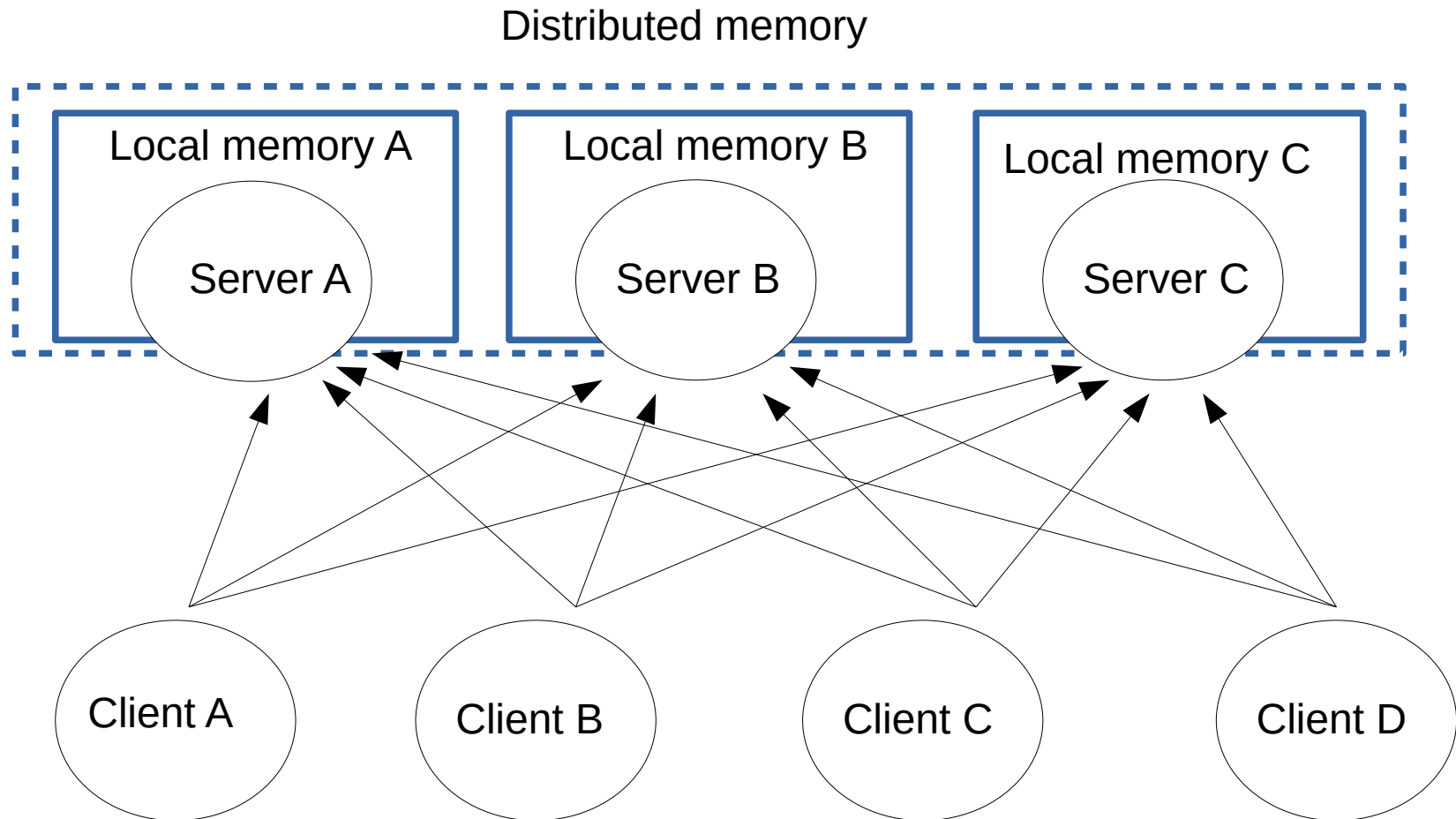
Πελάτης και διακομιστής: μοντέλο υπολογισμού 1

- Η κατανεμημένη μνήμη αποτελείται από το σύνολο των προστατευόμενων μεταβλητών. Οι αλλαγές στις τιμές αυτών των μεταβλητών υλοποιεί τη κατάσταση του κατανεμημένου υπολογισμού.
- Οι μεταβλητές αυτές δεν είναι μοιραζόμενες: είναι τοπικές σε κάποιο διακομιστή. Ο διακομιστής λειτουργεί ως μηχανισμός ταυτοχρονισμού για τις τοπικές του μεταβλητές.
- Πρωτόκολλο εφαρμογής: Οι διακομιστές δημοσιοποιούν τις διεπαφές των διαδικασιών που τροποποιούν τις προστατευόμενες τοπικές μεταβλητές.

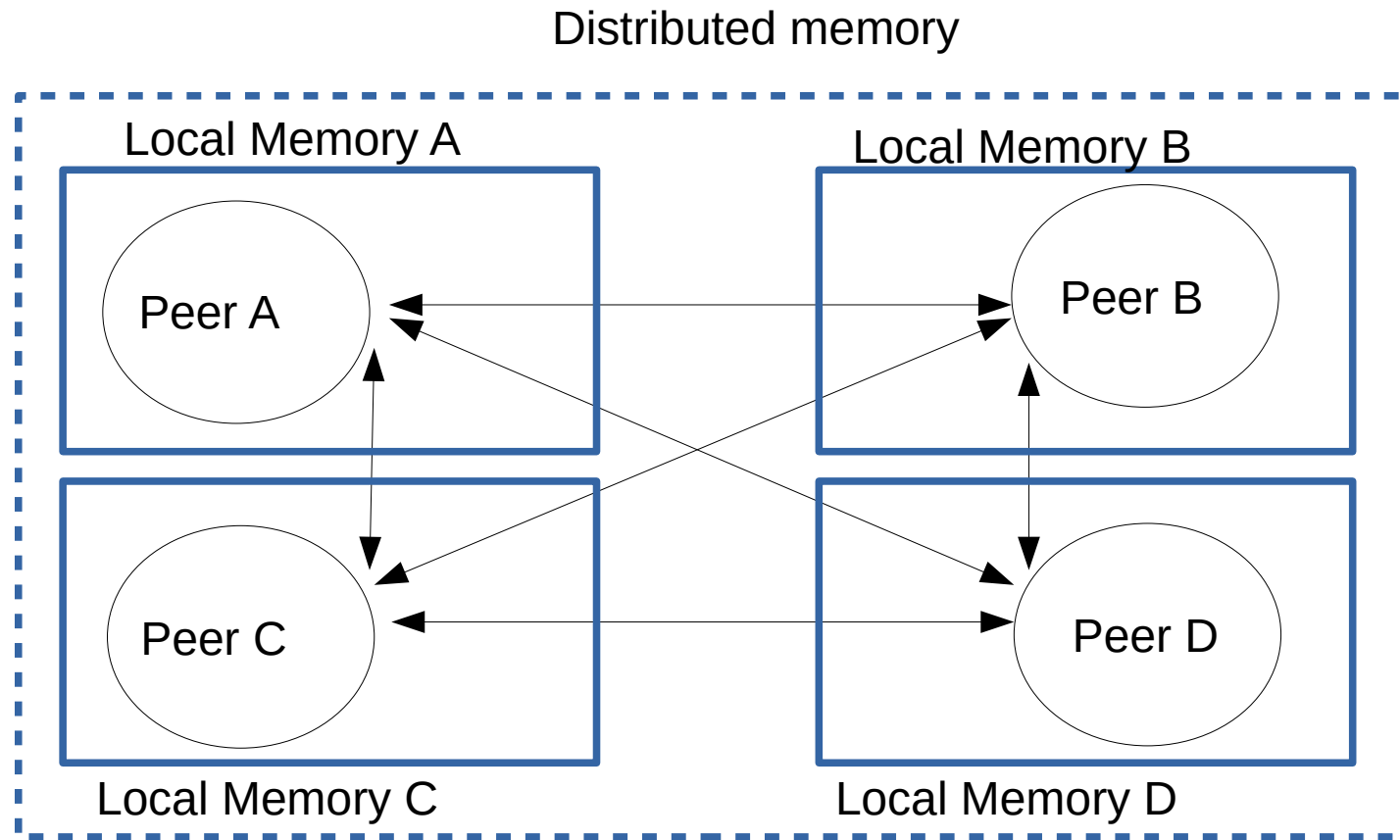
Πελάτης και διακομιστής: μοντέλο υπολογισμού 2

- Οι πελάτες γνωρίζουν το πρωτόκολλο της εφαρμογής, όπως και το τρόπο προσπέλασης στους διακομιστές.
- Οι πελάτες μπορούν να θεωρηθούν ως οντότητες που μπορεί να διαθέτουν τοπικές μεταβλητές αλλά αυτές δεν συνθέτουν την κατανεμημένη μνήμη, αλλά οι πελάτες επενεργούν (δηλαδή εκτελούν κώδικα του διακομιστή) επί των μεταβλητών των διακομιστών, μέσω μεταβίβασης μηνυμάτων.

Πελάτης και διακομιστής: μοντέλο υπολογισμού 3



Ομότιμες οντότητες



Μπορούμε να θεωρήσουμε ότι όλοι οι κόμβοι είναι πελάτες και διακομιστές.

Διακομιστής και επόπτης 1

Επόπτης: παθητικό αντικείμενο (passive object)

- Προστατεύει μεταβλητές
- Διαθέτει διαδικασίες αλλά δεν εκτελεί κώδικα
- Ο κώδικας εκτελείται από νήματα ή διεργασίες που καλούν τις διαδικασίες επόπτη.

Διακομιστής: ενεργητικό αντικείμενο (active object)

- Προστατεύει μεταβλητές
- Διαθέτει διαδικασίες και εκτελεί κώδικα
- Ο κώδικας εκτελείται από το διακομιστή με βάση μηνύματα που δέχεται από πελάτες (νήματα ή διεργασίες).

Διακομιστής και επόπτης 2

Με ποιο μηχανισμό ταυτοχρονισμού επιτυγχάνεται η προστασία των τοπικών μεταβλητών;

- Πλήρως ακολουθιακός διακομιστής: Σειριοποίηση της ταυτόχρονης επικοινωνίας.
- Διακομιστής οδηγούμενος από συμβάντα: Δυνατότητα πολυπλεξίας για διαχείριση ταυτόχρονης επικοινωνίας.
- Ταυτόχρονος διακομιστής: Ο διακομιστής εσωτερικά διαθέτει μηχανισμούς ταυτοχρονισμού μοιραζόμενης μνήμης (διαχείριση διεργασιών ή νημάτων).

Πελάτης διακομιστής: δικτυακή επικοινωνία 1

- Ο διακομιστής αναμένει εισερχόμενα μηνύματα - αιτήματα στα οποία συνήθως ανταποκρίνεται με την εκτέλεση κάποιας ενέργειας και αποστολή μηνύματος – απάντησης (request – reply).
- Το reply μπορεί να είναι απλή επιβεβαίωση ή να μην υπάρχει (μονόδρομη επικοινωνία).
- Ο πελάτης στέλνει αίτημα και συνήθως αναστέλλει την εκτέλεσή του μέχρι τη παραλαβή της απάντησης στο αίτημά του (ανασταλτική παραλαβή).

Πελάτης διακομιστής: δικτυακή επικοινωνία 2

- Ο πελάτης και ο διακομιστής μπορεί να είναι υλοποιημένα στην ίδια ή διαφορετικές γλώσσες προγραμματισμού ή / και να εκτελούνται σε διαφορετική αρχιτεκτονική / διαφορετικό λειτουργικό σύστημα.

Ασυμμετρία στην επικοινωνία

- Ο πελάτης πρέπει να γνωρίζει την υποδοχή του διακομιστή για να ζητήσει σύνδεση. Ξεκινά κατ' απαίτηση (αφού ξεκινήσει ο διακομιστής).
- Ο διακομιστής λειτουργεί πρώτος ή συνέχεια και αναμένει αιτήματα από οπουδήποτε. Όταν δεχτεί το αίτημα μαθαίνει τη διεύθυνση του πελάτη.

Πελάτης διακομιστής: δικτυακή επικοινωνία 3

Ασυμμετρία στην λειτουργικότητα

- Συνήθως αναπτύσσονται δύο ξεχωριστά προγράμματα.
- Χρησιμοποιούν το ίδιο πρωτόκολλο εφαρμογής αλλά από διαφορετική οπτική το καθένα: ο ένας ζητά και ο άλλος εξυπηρετεί.



Προγραμματισμός υποδοχών

Υποδοχές (sockets) 1

- Μια υποδοχή TCP/IP (internet socket) ορίζεται από μια διεύθυνση IP, ένα αριθμό θύρας καθώς και το πρωτόκολλο (TCP ή UDP).
- Η υποδοχή TCP/IP δεν αποτελεί τη δικτυακή σύνδεση καθ' αυτή αλλά το σημείο πρόσβασης στη δικτυακή σύνδεση (αναλογία: πρίζα και καλώδιο).
- Για το λειτουργικό σύστημα η υποδοχή TCP/IP είναι μια αναφορά (δείκτης) σε ένα buffer και εξασφαλίζει αμφίδρομη δικτυακή επικοινωνία host-to-host.
- Πχ 195.251.209.58:80/tcp είναι η υποδοχή του http (web) server www.it.uom.gr

Υποδοχές 2

- TCP

Προσανατολισμένο στη σύνδεση (αναλογία με τηλεφωνική σύνδεση). Δύο ρεύματα (streams) επικοινωνίας (εισερχόμενο, εξερχόμενο). Αξιόπιστη σύνδεση (τα μηνύματα μεταβιβάζονται με τη σειρά σε κάθε ρεύμα).

- UDP

Προσανατολισμένο στα πακέτα (αναλογία με τα sms). Αυτοδύναμα πακέτα, ανεξάρτητη αποστολή παραλαβή. Μη αξιόπιστη σύνδεση (πιθανή απώλεια πακέτων). Δεν εξετάζεται εδώ.

Θύρες (ports)

- Μια θύρα είναι μια λογική οντότητα (ένας προ-συμφωνημένος αριθμός) που, στα πλαίσια του λειτουργικού συστήματος και του TCP/IP, ορίζει μια δικτυακή εφαρμογή.
- Ο αριθμός θύρας (port number) είναι ο τρόπος που έχει το λειτουργικό σύστημα για να δρομολογεί τα μηνύματα της δικτυακής επικοινωνίας στη κατάλληλη διεργασία.
- Οι αριθμοί θύρας < 1024 , αλλά και αρκετοί άλλοι, λέγονται 'ευρέως γνωστοί' (well known) γιατί αντιστοιχούν σε γνωστές δικτυακές εφαρμογές, και γενικότερα ενδιάμεσο λογισμικό.

Ευρέως γνωστοί αριθμοί θύρας

```
$ cat /etc/services
# Network services, Internet style
...
# Updated from http://www.iana.org/assignments/port-numbers and
# other sources
tcpmux          1/tcp          # TCP port service multiplexer
echo            7/tcp
echo            7/udp
netstat         15/tcp
ftp             21/tcp
ssh             22/tcp          # SSH Remote Login Protocol
ssh             22/udp
telnet          23/tcp
smtp            25/tcp          mail
time            37/tcp          timserver
time            37/udp          timserver
nameserver      42/tcp          name      # IEN 116
http            80/tcp          www       # WorldWideWeb HTTP
http            80/udp          # HyperText Transfer Protocol
...
```

Berkeley Socket API

Primitive	Meaning
Socket	Create a new communication end point
Bind	Attach a local address to a socket
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Send	Send some data over the connection
Receive	Receive some data over the connection
Close	Release the connection

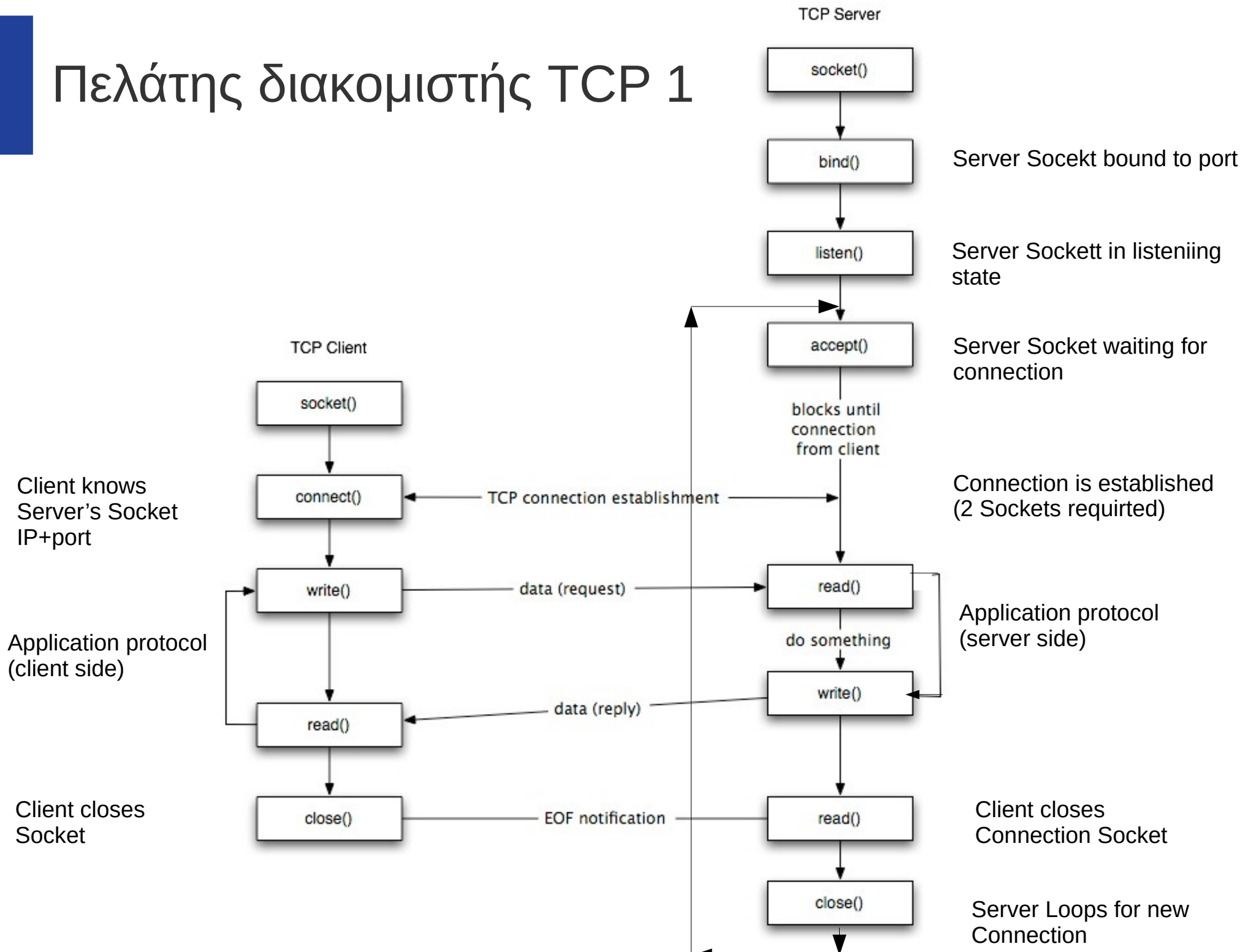
Υποδοχές TCP 1

- Μια υποδοχή μετά το Bind (IP και θύρα) μπορεί να είναι είτε σε κατάσταση αναμονής (Listening) ή σε κατάσταση σύνδεσης (Established), δηλαδή να αποδεχτεί αίτημα σύνδεσης (Connect).
- Υποδοχές σε κατάσταση αναμονής διαθέτουν οι διακομιστές, δηλαδή οι κατανεμημένες εφαρμογές που αναμένουν αιτήματα σύνδεσης. Λέγονται και υποδοχές διακομιστή.
- Για δεδομένο ζεύγος διεύθυνσης IP και θύρας μπορεί να υπάρχει μόνο μια υποδοχή σε αναμονή, η οποία είναι πάντα έτοιμη να δεχτεί αιτήματα σύνδεσης από πελάτες.

Υποδοχές TCP 2

- Μια υποδοχή πελάτη μετά το Bind προσπαθεί να κάνει Connect, δηλαδή να συνδεθεί (Established).
- Για δεδομένο ζεύγος διεύθυνσης IP και θύρας μπορεί να υπάρχουν πολλές ταυτόχρονες υποδοχές σε κατάσταση σύνδεσης (στη πράξη χρησιμοποιούν ψευδώνυμες θύρες).
- Μια δικτυακή σύνδεση αποτελείται από δύο υποδοχές, δηλαδή μια μοναδική τετράδα:
Τοπική διεύθυνση IP και αριθμός θύρας +
Απομακρυσμένη διεύθυνση IP και αριθμός θύρας
- Έτσι μια δικτυακή εφαρμογή μπορεί να έχει πολλές ταυτόχρονες συνδέσεις.

Πελάτης διακομιστής TCP 1



Πελάτης διακομιστής TCP 2

```
ClientSideProtocol {  
    MessageToServer prepareRequest() { ... }  
    processReply(messageFromServer) { ... }  
}
```

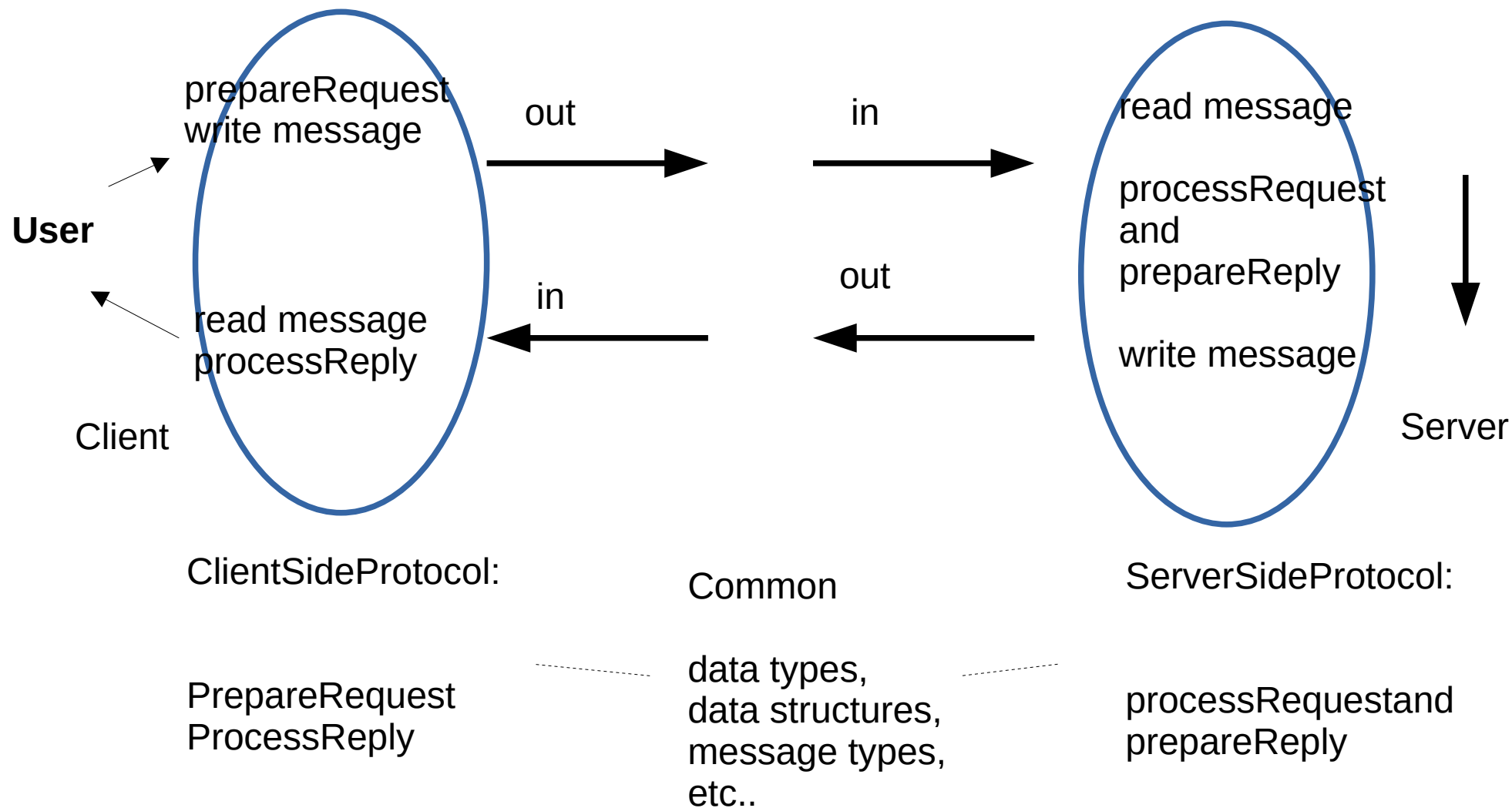
```
Client {  
    Socket();  
    connection = Connect (Server);  
    //connection between two sockets, Server address known  
    //connection establishes in, out streams  
  
    while (true) {  
        messageToServer = prepareRequest();  
        Write/Send (messageToServer, out);  
        if (message == exitMessage) break;  
        Read/Receive (messageFromServer, in);  
        processReply(messageFromServer);  
    }  
  
    Close Socket();  
}
```

Πελάτης διακομιστής TCP 3

```
ServerSideProtocol {
    messageToClient =
        processRequestandPrepareReply(messageFromClient) { ... }
}
Server
{
    Socket(Port); Bind(Socket); Listen(Socket);
    while (true) {
        connection = Accept (Socket);
        //accept returns an ephemeral connection Socket so
        //Server Socket remains free
        //connection establishes in, out streams

        while (true) {
            Receive/Read (messageFromClient, in);
            messageToClient =
                processRequestandPrepareReply(messageFromClient);
            if (messageToClient == exitMessage) break;
            Send/Write (messageToClient, outStream);
        }
        close Socket();
    }
}
```

Πελάτης διακομιστής TCP 4



Πρωτόκολλα εφαρμογής 1

- Οι κανόνες που διέπουν την επικοινωνία μεταξύ δύο οντοτήτων του ίδιου επιπέδου (εδώ πελάτη και διακομιστή) . Περιλαμβάνουν:
- τύπους και δομές δεδομένων
- τύπους μηνυμάτων
- κανόνες αιτήματος – απόκρισης (ζεύγη μηνυμάτων)
- σφάλματα, διαχείριση σφαλμάτων
- οι υλοποιήσεις των συναρτήσεων μπορεί να βρίσκονται σε ξεχωριστά αρχεία / κλάσεις.
- αποφεύγουμε τα πρωτόκολλα με καταστάσεις!

Πρωτόκολλα εφαρμογής 2

```
ClientSideProtocolCalculator
{
    // data types, data structures

    int a,b,r;
    char op; // +, -, *, /
    int err_code;
    data ...

    // message types

    // requestMessage (string): op <space> a <space> b
    // exitMessage (string): !
    // replyMessage (string): R <space> r
    // errorMessage (string): E <space> err_code
```



```
requestMessage PrepareRequest () {  
    done = false;  
    while (!done)  
        promptUser();  
        data = readUserInput();  
        done = checkData(data);  
    }  
    request = buildRequestMessage(data);  
    return(request);  
}
```

Πρωτόκολλα εφαρμογής 3

```
ProcessReply (reply) {  
    data = getReplyData(reply)  
    displayDatatoUser(data);  
}
```

```
// helper functions ..
```

```
}
```

Πρωτόκολλα εφαρμογής 4

```
ServerSideProtocolCalculator  
{
```

```
    // data types, data structures, message types
```

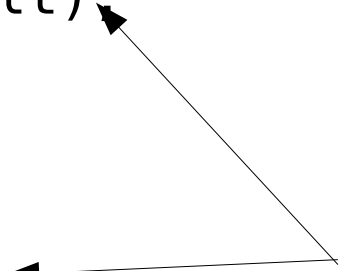
```
    ProcessRequestandProduceReply (request) {
```

```
        data = getRequestData(request);  
        result = doServerComputation(data);  
        reply = buildReplyMessage(result);  
        return(reply);
```

```
    }
```

```
    // server computing functions ..
```

```
}
```



Υλοποίηση υπολογισμού
Πιθανώς ξεχωριστό
αρχείο ή κλάση

...

Πρωτόκολλα εφαρμογής 5

```
doServerComputation(data) {  
    switch (data.op) {  
        case '+' : result.op = 'R';  
                    result.r = data.a + data.b;  
                    break;  
        case '-' : result.op = 'R';  
                    result.r = data.a - data.b;  
                    break;  
        case '*' : result.op = 'R';  
                    result.r = data.a * data.b;  
                    break;  
        case '/' : result.op = 'R';  
                    result.r = data.a / data.b;  
                    break;  
        case '!' : result.op = '!';  
                    result.r = 0;  
                    break;  
        default: //error handling...  
    }  
}
```

}

Κατάσταση (state) διακομιστή 1

- Όταν μεταξύ διαδοχικών αιτημάτων ο διακομιστής διατηρεί `μνήμη`, τότε λέμε ότι ο διακομιστής έχει καταστάσεις (state-full).
- Όταν κάθε ζεύγος αίτημα - απάντηση είναι ανεξάρτητο από τα υπόλοιπα, τότε λέμε ότι ο διακομιστής δεν έχει καταστάσεις (state-less).
- State-full επαναληπτική λειτουργία:
αίτημα1 -> επεξεργασία1 -> απάντηση1 ->
αίτημα2 -> επεξεργασία2 -> απάντηση2 ...
η επεξεργασία2 διατηρεί πληροφορίες από την επεξεργασία 1 (πχ αθροιστής).

Κατάσταση (state) διακομιστή 2

- State-less επικοινωνία:

Το αίτημα (ή και η απάντηση) στέλνεται σε ένα μήνυμα. Απλό πρωτόκολλο εφαρμογής, πιθανή ανάγκη πλεονασμού (επανάληψης) πληροφορίας.

- State-full επικοινωνία:

Το αίτημα (ή και η απάντηση) στέλνεται σε σειρά μηνυμάτων. Πιο σύνθετο πρωτόκολλο εφαρμογής, συνήθως μηχανή πεπερασμένων καταστάσεων, πιθανώς ταχύτερη απόκριση σε σύνθετα μηνύματα.

```
ServerSideProtocolCalculator
{
    ... acc = 0; //state

    ProcessRequestandProduceReply (request) {
        data = getRequestData(request);
        result = doServerComputation(data);
        reply = buildReplyMessage(data);
        return(reply);
    }
    ...

    switch (data.op) {
        ...
        case 'M' : result.op = 'R';
                    result.r = acc;
                    break;
        case 'M0' : result.op = 'R';
                    acc = 0;
                    result.r = acc;
                    break;
        case 'M+' : result.op = 'R';
                    result.r = data.a + data.b + acc;
                    break;
        ...

        default: //error handling...
    }
}
```

Κατάσταση
(state)
διακομιστή:
calculator 1

```
requestMessage0 (string): op  
requestMessage1 (string): a  
requestMessage2 (string): b
```

```
States: OP2, OP1, OP2, OP3
```

```
state = OP0  
switch (state){  
    case OP0 : receive requestMessage0  
                if ok state = OP1  
                else if exit state = OP0  
                    else send errMessage0  
                break  
    case OP1 : receive requestMessage1  
                if ok state = OP2  
                else if exit state = OP0  
                    else send errMessage1  
                break  
    case OP2 : receive requestMessage2  
                if ok state = OP3  
                else if exit state = OP0  
                    else send errMessage2  
                break  
    case OP3 : calculate result // doServerComputation  
                send result  
                state = OP0  
                break  
}
```

Κατάσταση
(state)
διακομιστή:
calculator 2

Έλξη ή Ώθηση (Pull or Push)

Μετά τη σύνδεση, μπορεί

- Να ξεκινά πρώτος ο πελάτης με αίτημα και να περιμένει απάντηση (έλξη)
- Ο διακομιστής να στέλνει τα δεδομένα (την απάντηση) χωρίς σχετικό αίτημα (ώθηση)
- Ο διακομιστής να ενημερώνει τον πελάτη ότι έχει δεδομένα γι' αυτόν (ειδοποίηση) και να ακολουθεί έλξη
- Ο πελάτης να στέλνει επιβεβαίωση παραλαβής (acknowledgement) μετά από έλξη ή ώθηση



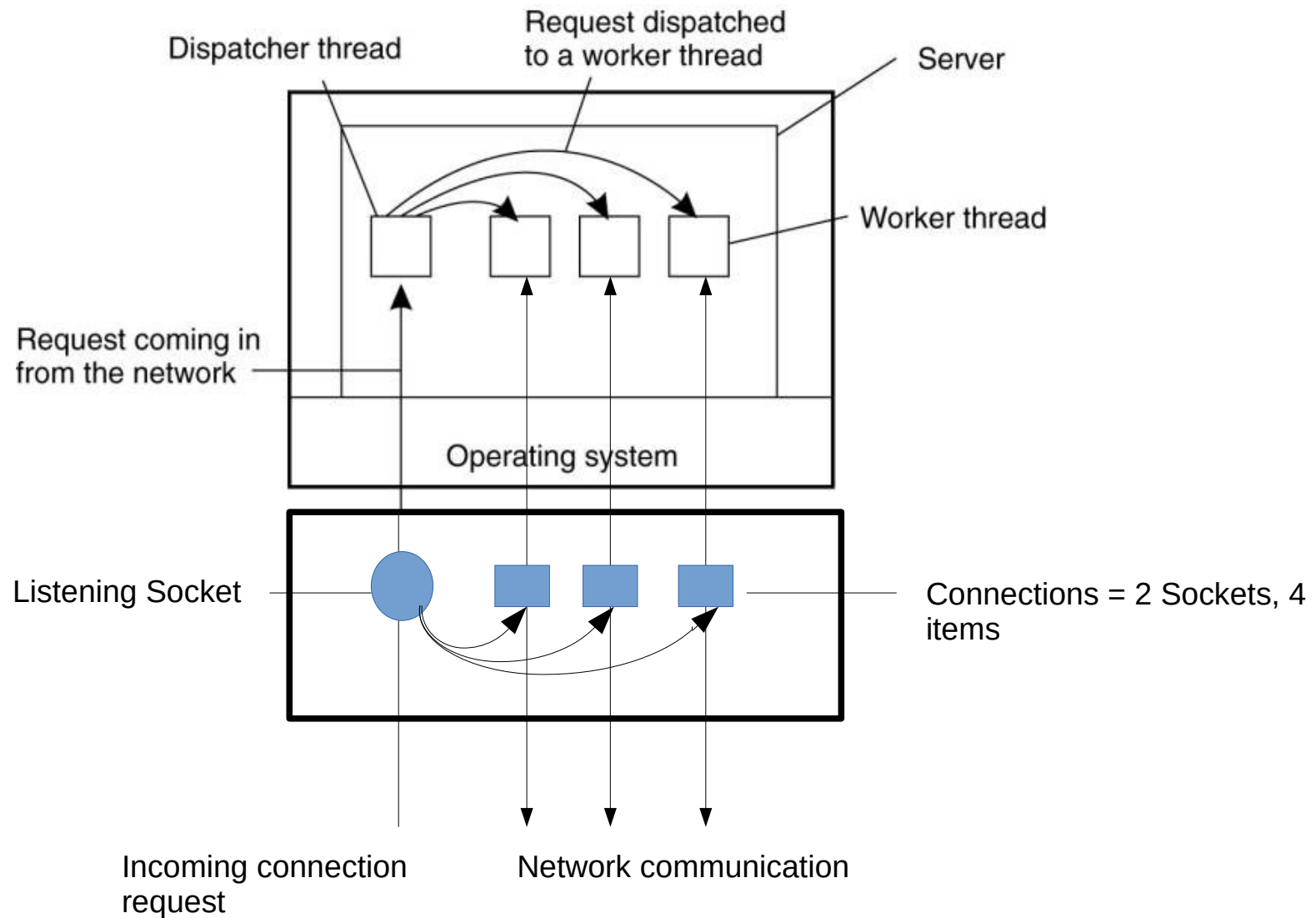
Ταυτόχρονος διακομιστής

Μηχανισμοί ταυτοχρονισμού στον διακομιστή

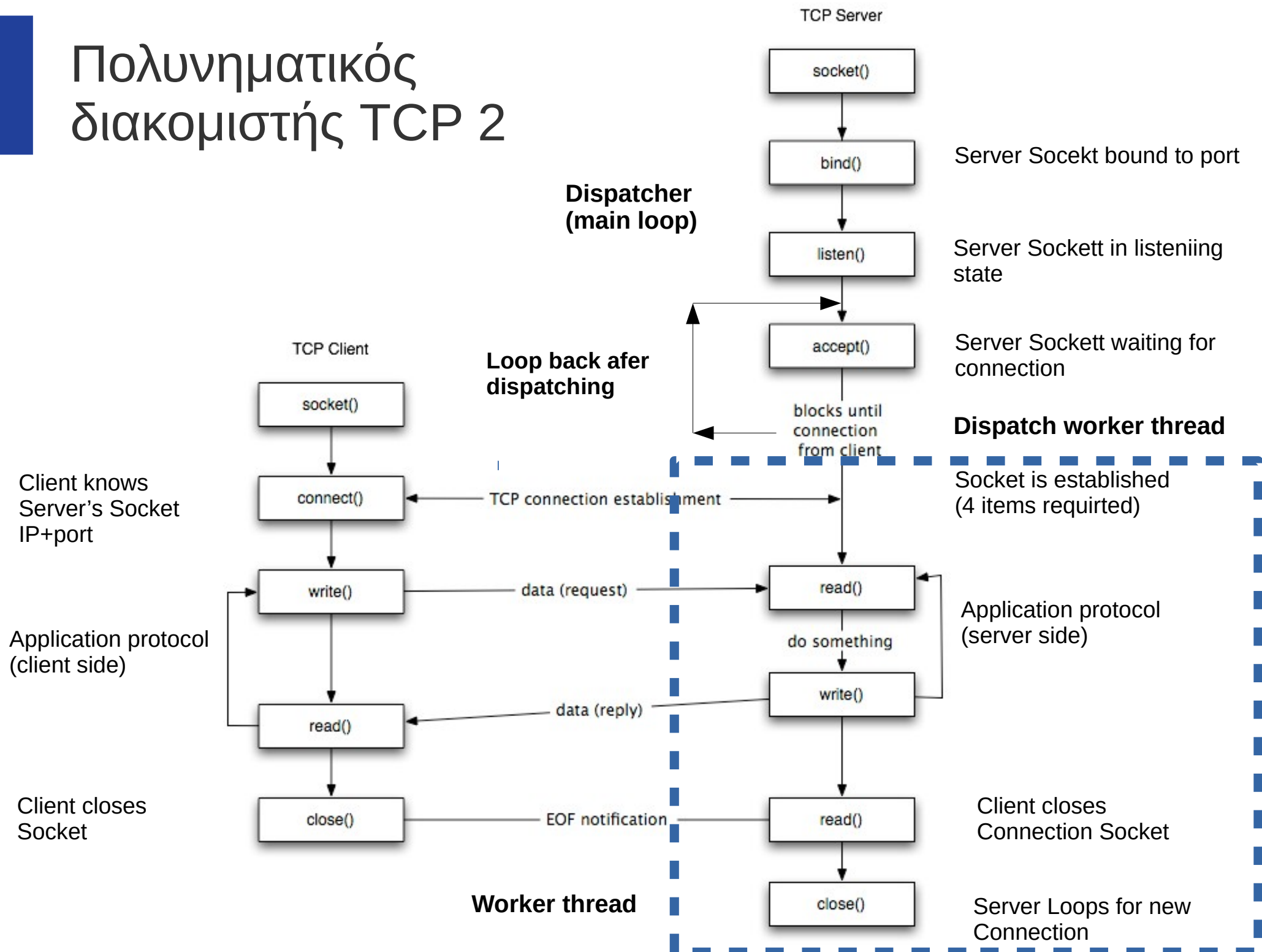
- Πλήρως ακολουθιακός διακομιστής: Σειριοποίηση της ταυτόχρονης επικοινωνίας.
- Διακομιστής οδηγούμενος από συμβάντα: Δυνατότητα πολυπλεξίας για διαχείριση ταυτόχρονης επικοινωνίας.*
- Ταυτόχρονος διακομιστής: Ο διακομιστής εσωτερικά διαθέτει μηχανισμούς ταυτοχρονισμού μοιραζόμενης μνήμης (διαχείριση διεργασιών ή νημάτων).

* ΔΕΝ καλύπτεται εδώ

Πολυνηματικός διακομιστής TCP 1



Πολυνηματικός διακομιστής TCP 2



Πολυνηματικός διακομιστής TCP 3

```
MultiThreadedServerTCP
```

```
{
```

```
    Socket(Port); Bind(Socket); Listen(Socket);
```

```
    while (true) {
```

```
        connection = Accept (Socket);
```

```
        //accept returns an ephemeral connection Socket so
```

```
        //Server Socket remains free
```

```
        //connection establishes in, out streams
```

```
        dispatch serverThread (connection);
```

```
        //dispatch = create and run a server
```

```
}
```

Πλήθος εξυπηρετούμενων πελατών 1

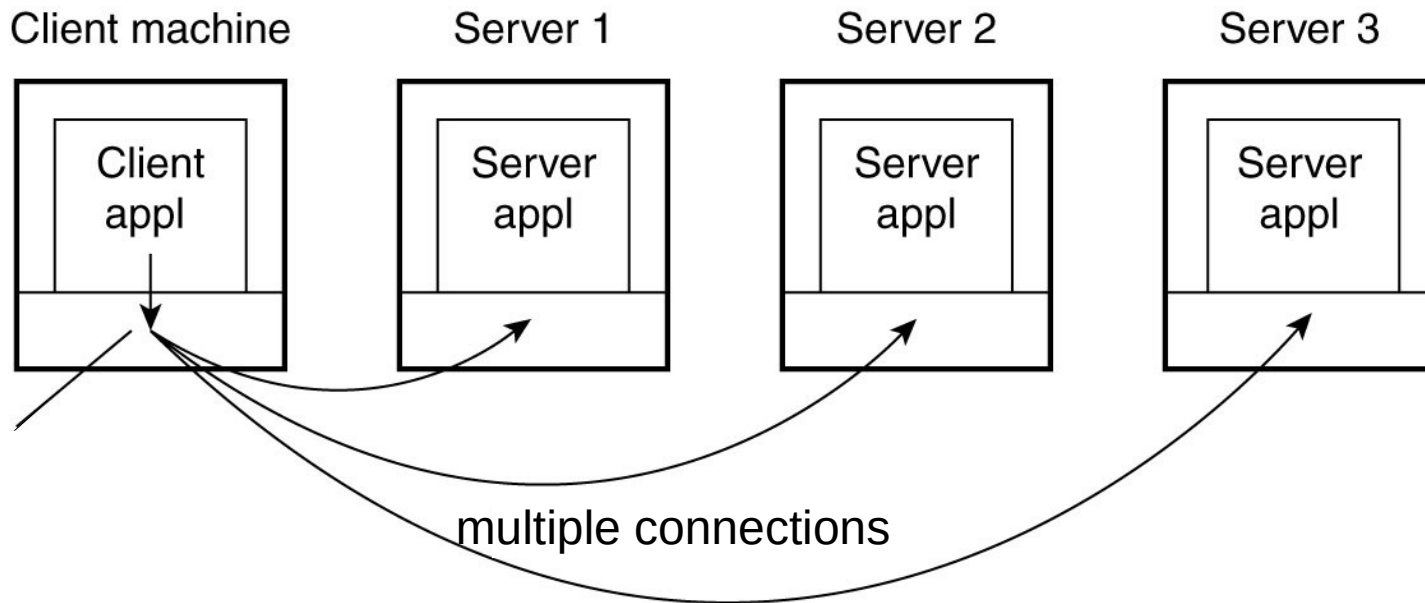
- Ο πολυνηματικός διακομιστής δημιουργεί ένα νήμα για κάθε αίτημα σύνδεσης (δηλαδή, εξυπηρετούμενο πελάτη). Κάθε νήμα χρειάζεται ορισμένους πόρους του συστήματος για τη λειτουργία του.
- Ένα μεγάλο πλήθος ταυτόχρονα εξυπηρετούμενων πελατών μπορεί να οδηγήσει σε αύξηση του χρόνου απόκρισης του διακομιστή ή ακόμη και σε κατάρρευση.
- Αυτό το φαινόμενο μπορεί να οφείλεται είτε απλή συγκυρία (πχ απότομη αύξηση επισκεψιμότητας) ή σε κακόβουλη ενέργεια (πχ DOS/DDOS attacks).

Ουρά εισερχόμενων αιτημάτων (pre-threading)

Ισοδυναμεί με παραγωγό καταναλωτή και δυναμική κατανομή φόρτου.

- Ο διακομιστής κατά την εκκίνησή του δημιουργεί ένα συγκεκριμένο αριθμό νημάτων.
- Τα εισερχόμενα αιτήματα σύνδεσης τοποθετούνται σε μια ουρά (buffer) από connection tasks (put, enqueue).
- Κάθε νήμα εξυπηρετεί (get, dequeue) το connection task στην κεφαλή της ουράς.
- Όταν ένα νήμα τελειώσει με την εξυπηρέτηση ενός πελάτη (connection task), δηλαδή ο πελάτης κλείσει τη σύνδεση, τότε το νήμα λαμβάνει άλλο connection task από την ουρά.
- Όταν δεν υπάρχουν συνδέσεις τα νήματα είναι άεργα (idle)

Πολυνηματικός πελάτης TCP 1



Δημιουργία νημάτων στο πελάτη

- Αρχικός σχεδιασμός, για παράδειγμα διαχείριση GUI
- Με βάση γεγονότα (events) του χρήστη, για παράδειγμα πολλαπλές συνδέσεις
- Παραπέρα λειτουργικότητα μέσα σε μια σύνδεση, για παράδειγμα μεταφόρτωση

Πολυνηματικός πελάτης TCP 2

```
MultiThreadedClient
```

```
{  
    draw_GUI();  
  
    while (true) {  
        Ui = getUserInput();  
        switch (Ui.operation){  
            case CreateNewConnection  
                dispatch clientThread (Ui.data);  
                // create thread to manage  
                // new Client connection  
            case CloseConnection  
                stop clientThread (Ui.data);  
                // close Client connection  
                // kill thread  
            case ...  
        }  
        refresh_GUI();  
    }  
}
```

Πολυνηματικός πελάτης TCP 3

- Γρήγορη επιστροφή στο κύριο βρόχο του πελάτη.
- Τα νήματα εξυπηρέτησης της κάθε σύνδεσης εκτελούνται αυτόνομα, ο πελάτης δεν περιμένει την επιστροφή τους.
- Επικοινωνία με διακοπές: ο πελάτης τερματίζει ένα νήμα
- Επικοινωνία με ασύγχρονο I/O- callback(): ο πελάτης συνεχίζει στο κύριο βρόχο μέχρι να ολοκληρωθεί κάποια λειτουργία ενός νήματος.

Καθυστερήσεις επικοινωνίας

- Στον πολυνηματικό διακομιστή, τα νήματα την περισσότερη ώρα αναμένουν την επικοινωνία με τον πελάτη. Συνήθως ο χρόνος εξυπηρέτησης (service time) είναι πολύ μικρότερος.
- Στον πολυνηματικό πελάτη, τα νήματα αναμένουν είτε την επικοινωνία με τον διακομιστή, είτε την επικοινωνία με τον χρήστη. Συνήθως ο χρόνος που εκτελεί ο πελάτης μια λειτουργία είναι πολύ μικρότερος.
- Επομένως τα νήματα και στις δύο περιπτώσεις ουσιαστικά υπολειτουργούν.

Λύσεις

- Ασύγχρονη κλήση:

Async Calls, Futures and Promises. Event-driven, Reactive προγραμματισμός.

Πχ Java completable futures

- Υβριδικά νήματα:

User-Level Threads που μοιράζονται ένα pool από Kernel-Level Threads. Αναστολή στο I/O και αφύπνιση όταν η επικοινωνία είναι έτοιμη.

Πχ Java virtualThreads



Παράλληλος και Κατανεμημένος Υπολογισμός I

Υποδοχές στη Java

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025

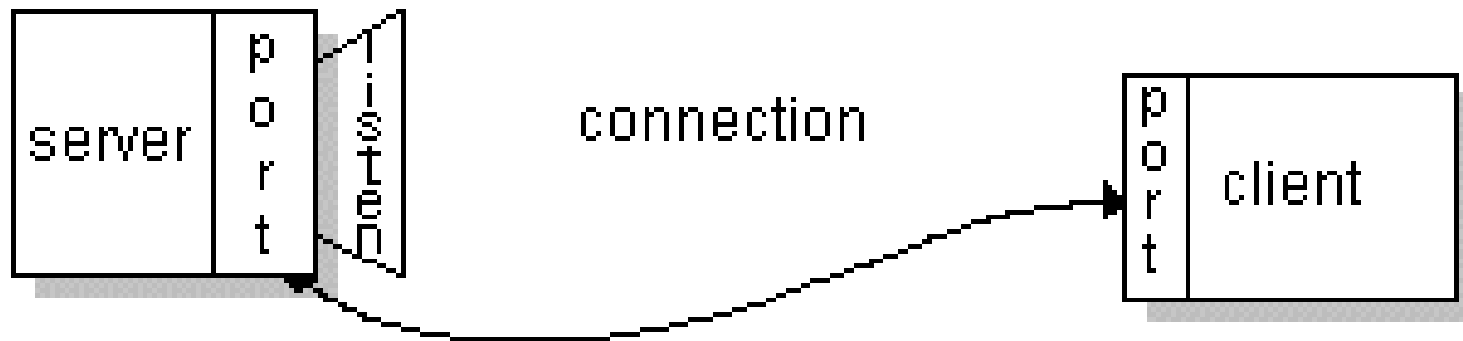


Προγραμματισμός υποδοχών

Socket και Server Socket -TCP



Η ServerSocket “ακούει” σε συγκεκριμένη θύρα (port) του διακομιστή (server) σε αναμονή αιτήματος σύνδεσης από την υποδοχή (socket) κάποιο πελάτη (client)



Ως απάντηση στο αίτημα σύνδεσης η ServerSocket παράγει μια υποδοχή (socket) από τη πλευρά του πελάτη η οποία συνδέεται με αυτή του πελάτη και επιτυγχάνεται η σύνδεση.

Απλός TCP Server 1

```
import java.net.*;
import java.io.*;

public class SimpleServerTCP {

    //Server's port
    private static final int PORT = 1234;

    public static void main(String args[]) throws IOException {

        //Establish listening server socket
        ServerSocket connectionSocket = new ServerSocket(PORT);
        System.out.println("Server is listening to port: " + PORT);

        //Wait for connection and produce actual socket
        Socket dataSocket = connectionSocket.accept();
        System.out.println("Received request from " +
                           dataSocket.getInetAddress());

        ...
    }
}
```

Code/Simple-client-server-tcp-protocol

Απλός TCP Server 2

```
...  
//Set up input and output streams  
InputStream is = dataSocket.getInputStream();  
BufferedReader in = new BufferedReader(new  
InputStreamReader(is));  
  
OutputStream os = dataSocket.getOutputStream();  
PrintWriter out = new PrintWriter(os,true);  
  
//Receive request message, process it and send reply back  
String inmsg, outmsg;  
inmsg = in.readLine();  
ServerProtocol app = new ServerProtocol();  
outmsg = app.processRequest(inmsg);  
out.println(outmsg);  
  
dataSocket.close();  
connectionSocket.close();  
System.out.println("Data socket closed");  
}  
}
```

Παρατηρήσεις 1

- Εισαγωγή βιβλιοθηκών `java.net`, `java.io`
- `Server Socket`: ισοδύναμο με την ακολουθία `Socket(); Bind(); Listen();`
- Σε άλλες γλώσσες ρητή κλήση των συναρτήσεων (πχ `C`, `Python`)
- Ρητά ονόματα στα `Net I/O Streams`: δυνατότητα επιλογής διαμορφωμένης Εισόδου/Εξόδου.
- Σε άλλες γλώσσες τα `Net I/O Streams` είναι ρεύματα χαρακτήρων και η διαμόρφωση / επεξεργασία γίνεται μετά τη παραλαβή.
- Τυπική κωδικοποίηση χαρακτήρων: 8-bit ASCII.

Παρατηρήσεις 2

- Οι κλήσεις σε συναρτήσεις Net I/O συνήθως περικλείονται σε

```
try { ....} catch { IOException (e)...}
```

ή άλλες εξαιρέσεις όπως `UnknownHostException`, `ConnectionException` ανάλογα με τη κλήση.

- Όπου είναι δυνατό χρησιμοποιούμε το `throws IOException`

για να μειώσουμε τη πολυπλοκότητα του κώδικα, εκτός αν απαιτείται λεπτομερής διαχείριση εξαιρέσεων.

```
import java.net.*;
import java.io.*;
```

Απλός TCP Client

1

```
public class SimpleClientTCP {
```

```
    //Client should know server's address/name
```

```
    private static final String HOST = "localhost";
```

```
    private static final int PORT = 1234;
```

```
    public static void main(String args[]) throws IOException {
```

```
        //Try to connect to server
```

```
        //InetAddress address = InetAddress.getByName(HOST);
```

```
        Socket dataSocket = new Socket(HOST, PORT);
```

```
        //Set up input and output streams
```

```
        InputStream is = dataSocket.getInputStream();
```

```
        BufferedReader in = new BufferedReader(new  
                                                    InputStreamReader(is));
```

```
        OutputStream os = dataSocket.getOutputStream();
```

```
        PrintWriter out = new PrintWriter(os,true);
```

```
        System.out.println("Connection to " + HOST + " established");
```

```
        ...
```

Απλός TCP Client 2

...

```
//Send request and then receive and process reply  
String inmsg, outmsg;  
ClientProtocol app = new ClientProtocol();  
outmsg = app.prepareRequest();  
out.println(outmsg);  
inmsg = in.readLine();  
app.processReply(inmsg);
```

```
dataSocket.close();  
System.out.println("Data Socket closed");
```

```
}
```

```
}
```

Απλό πρωτόκολλο εφαρμογής: server

```
import java.net.*;
import java.io.*;

public class ServerProtocol {

    public String processRequestandProduceReply(String theInput) {

        String theOutput;
        System.out.println("Received message from client: " + theInput);

        //Processing of input occurs here (request – reply)..
        //Can include any type of computation or even user input
        //Can include several methods, classes

        theOutput = theInput;
        System.out.println("Send message to client: " + theOutput);
        return theOutput;
    }
}
```

Απλό πρωτόκολλο εφαρμογής: client

```
import java.net.*;
import java.io.*;

public class ClientProtocol {

    BufferedReader user = new BufferedReader(new
                                                InputStreamReader(System.in));

    //Request comes from user input
    public String prepareRequest() throws IOException {

        System.out.print("Enter message to send to server:");
        String theOutput = user.readLine();
        return theOutput;
    }

    //Reply goes to user screen
    public void processReply(String theInput) throws IOException {

        System.out.println("Message received from server: " + theInput);
    }
}
```

Echo Server TCP

```
...  
public class EchoServerTCP {  
    ...  
    private static final String EXIT = "CLOSE";  
  
    public static void main(String args[]) throws IOException {  
        ...  
        String inmsg, outmsg;  
        inmsg = in.readLine();  
        ServerProtocol app = new ServerProtocol();  
        outmsg = app.processRequest(inmsg);  
        while(!outmsg.equals(EXIT)) {  
            out.println(outmsg);  
            inmsg = in.readLine();  
            outmsg = app.processRequest(inmsg);  
        }  
  
        dataSocket.close();  
        System.out.println("Data socket closed");  
        ...  
    }  
}
```

Code/EchoServerTCP

Echo Client TCP

```
...  
public class EchoClientTCP {  
    ...  
    private static final String EXIT = "CLOSE";  
  
    public static void main(String args[]) throws IOException {  
  
        ...  
        String inmsg, outmsg;  
        ClientProtocol app = new ClientProtocol();  
        outmsg = app.prepareRequest();  
        while(!outmsg.equals(EXIT)) {  
            out.println(outmsg);  
            inmsg = in.readLine();  
            app.processReply(inmsg);  
            outmsg = app.prepareRequest();  
        }  
        out.println(outmsg);  
  
        dataSocket.close();  
        System.out.println("Data Socket closed");  
        ...  
    }  
}
```

Επαναληπτικός Echo Server TCP

...

```
public class IterativeEchoServerTCP {  
    ...  
  
    public static void main(String args[]) throws IOException {  
  
        ServerSocket connectionSocket = new ServerSocket(PORT);  
  
        while (true) {  
  
            System.out.println("Server is listening to port: " + PORT);  
            Socket dataSocket = connectionSocket.accept();  
  
            //Remaining Echo TCP server code here ...  
            //After closing socket back to listening  
  
        }  
    }  
}
```

Code/IterativeEchoServerTCP

Πολυνηματικός Echo Server TCP 1

```
import java.net.*;
import java.io.*;

public class MultithreadedEchoServerTCP {
    private static final int PORT = 1234;

    public static void main(String args[]) throws IOException {

        ServerSocket connectionSocket = new ServerSocket(PORT);

        while (true) {

            System.out.println("Server is listening to port: " + PORT);
            Socket dataSocket = connectionSocket.accept();
            System.out.println("Received request from " +
                               dataSocket.getInetAddress());

            ServerThread sthread = new ServerThread(dataSocket);
            sthread.start();

        }
    }
}
```

```
import java.io.*;
import java.net.*;
```

```
class ServerThread extends Thread
{
```

```
    private Socket dataSocket;
    private InputStream is;
    private BufferedReader in;
    private OutputStream os;
    private PrintWriter out;
    private static final String EXIT = "CLOSE";
```

```
    public ServerThread(Socket socket)
    {
```

```
        dataSocket = socket;
```

```
        try {
```

```
            is = dataSocket.getInputStream();
```

```
            in = new BufferedReader(new InputStreamReader(is));
```

```
            os = dataSocket.getOutputStream();
```

```
            out = new PrintWriter(os,true);
```

```
        }
```

```
        catch (IOException e) {
```

```
            System.out.println("I/O Error " + e);
```

```
        }
```

```
    }
```

```
    ...
```

Πολυνηματικός Echo Server TCP 2

Πολυνηματικός Echo Server TCP 3

```
...  
public void run()  
{  
    String inmsg, outmsg;  
  
    try {  
        inmsg = in.readLine();  
        ServerProtocol app = new ServerProtocol();  
        outmsg = app.processRequest(inmsg);  
        while(!outmsg.equals(EXIT)) {  
            out.println(outmsg);  
            inmsg = in.readLine();  
            outmsg = app.processRequest(inmsg);  
        }  
  
        dataSocket.close();  
        System.out.println("Data socket closed");  
    } catch (IOException e) {  
        System.out.println("I/O Error " + e);  
    }  
}
```

Code/Echo-client-server-tcp/MultithreadedEchoServerTCP